

Questions

1. HTML Document Structure

Which tag should contain the visible content of the page?

- A. <head>
 - B. <body>
 - C. <meta>
 - D. <title>
-

2. Required Form Field

Which input is required before the form can be submitted?

- A. <input type="text" must>
 - B. <input type="text" required>
 - C. <input type="text" validate>
 - D. <input type="text" needed>
-

3. External JavaScript File

Your JS file is located here:

js/app.js

Which line correctly connects it to index.html?

- A. <script href="./js/app.js"></script>
 - B. <javascript src="./js/app.js"></javascript>
 - C. <script src="./js/app.js"></script>
 - D. <link rel="script" href="./js/app.js">
-

4. Script Loading Problem

This script is inside the <head>:

<script src="./js/app.js"></script>

And the JS file contains:

```
const title = document.querySelector("#title");  
title.textContent = "Hello";
```

Sometimes `title` is `null`. Why?

- A. The script may run before the HTML element exists
 - B. `querySelector` cannot select IDs
 - C. `textContent` works only on buttons
 - D. JavaScript cannot run from the `<head>`
-

5. Best Fix for Script Loading

What is a good fix for the previous problem?

- A. Put the script after the HTML or use `defer`
 - B. Rename the file to `script.html`
 - C. Add `!important` to the JavaScript
 - D. Use `position: absolute`
-

6. CSS Class Selector

Which CSS selector targets this element?

```
<p class="error">Invalid password</p>
```

- A. `#error`
 - B. `.error`
 - C. `p#error`
 - D. `error`
-

7. CSS ID Selector

Which CSS selector targets this element?

```
<section id="profile"></section>
```

- A. .profile
 - B. profile
 - C. #profile
 - D. section.profile
-

8. CSS Cascade

Given:

```
p {  
  color: red;  
}
```

```
p {  
  color: blue;  
}
```

What color will the paragraph be?

- A. Red
 - B. Blue
 - C. Both red and blue
 - D. No color
-

9. box-sizing: border-box

Given:

```
.card {  
  width: 300px;  
  padding: 20px;  
  border: 5px solid black;  
  box-sizing: border-box;  
}
```

What is the final rendered width?

- A. 300px
- B. 340px
- C. 350px
- D. 250px

10. Flex Direction

What does this CSS do?

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

- A. Places children in a row
- B. Places children vertically
- C. Hides all children
- D. Makes the container fixed

11. JavaScript Equality

What is the result?

```
console.log(5 == "5");  
console.log(5 === "5");
```

- A. true, true
- B. false, false
- C. true, false
- D. false, true

12. Array Filter

What does this code return?

```
const nums = [1, 2, 3, 4];  
  
const result = nums.filter(num => num > 2);
```

- A. [1, 2]
- B. [3, 4]
- C. [true, true, false, false]
- D. 4

13. Template Literals

Which option correctly uses a template literal?

- A. "Hello \${name}"
 - B. 'Hello \${name}'
 - C. `Hello \${name}`
 - D. "Hello " + \${name}
-

14. Event Listener

Which code correctly listens for a button click?

A.

```
button.on("click", () => {});
```

B.

```
button.addEventListener("click", () => {});
```

C.

```
button.listen("click", () => {});
```

D.

```
button.clickEvent(() => {});
```

15. JSON.stringify

Why do we use `JSON.stringify()` in a fetch POST request?

- A. To convert a JavaScript object into a JSON string
 - B. To convert JSON into CSS
 - C. To stop the request from being sent
 - D. To automatically create a database
-

16. Wrong HTTP Method

Client code:

```
fetch("/api/users", {  
  method: "POST"  
});
```

Server code:

```
app.get("/api/users", (req, res) => {  
  res.json([]);  
});
```

The request fails. What is the problem?

- A. The client sends POST, but the server only has GET
 - B. /api/users is an illegal route name
 - C. fetch only supports GET
 - D. Express routes cannot return arrays
-

17. Missing Response

What is wrong with this route?

```
app.get("/api/test", (req, res) => {  
  console.log("Request received");  
});
```

- A. The server receives the request but never sends a response
 - B. console.log crashes Express
 - C. GET requests cannot use callbacks
 - D. The route must include async
-

18. Static Files

You have this structure:

```
server.js  
public/  
  index.html  
  style/
```

main.css

Which line allows Express to serve files from the public folder?

- A. `app.use(express.json())`
 - B. `app.use(express.static("public"))`
 - C. `app.use(express.public())`
 - D. `app.listen("public")`
-

19. Client-Server Responsibility

Where should password checking against a database happen?

- A. Only in client-side JavaScript
 - B. On the server
 - C. In CSS
 - D. In localStorage
-

20. Reading URL Query

Request:

GET /api/products?category=games

Route:

```
app.get("/api/products", (req, res) => {  
  res.json(req.query.category);  
});
```

What is returned?

- A. "games"
 - B. "products"
 - C. undefined
 - D. "/api/products"
-

21. Route Parameter

Which route matches this request?

GET /api/users/15

- A. `app.get("/api/users", ...)`
 - B. `app.get("/api/users/:id", ...)`
 - C. `app.get("/api/users?id=15", ...)`
 - D. `app.get("/api/:users/id", ...)`
-

22. Environment Variables

Why should `.env` usually not be committed to GitHub?

- A. It usually contains sensitive or machine-specific values
 - B. It is too large
 - C. GitHub does not support text files
 - D. Node.js cannot read committed files
-

23. .gitignore

Which file should usually include `node_modules` and `.env`?

- A. `package.json`
 - B. `.gitignore`
 - C. `index.html`
 - D. `server.js`
-

24. package.json

What is the main purpose of `package.json`?

- A. It stores the HTML structure
 - B. It stores project metadata, scripts, and dependencies
 - C. It stores all uploaded images
 - D. It replaces the server file
-

25. npm install

After cloning a Node.js project from GitHub, why do we usually run `npm install`?

- A. To create `index.html`
 - B. To install dependencies listed in `package.json`
 - C. To start the browser
 - D. To delete `node_modules`
-

26. Wrong Content-Type

Client sends JSON:

```
fetch("/api/login", {  
  method: "POST",  
  headers: {  
    "Content-Type": "text/plain"  
  },  
  body: JSON.stringify({ username: "dana" })  
});
```

Server uses:

```
app.use(express.json());
```

Why might `req.body` not be parsed as expected?

- A. The request says the body is `text/plain`, not `application/json`
 - B. `express.json()` only works with GET
 - C. JSON cannot contain strings
 - D. The route must be called `/json`
-

27. CORS

The browser blocks a request from `http://localhost:3000` to `http://localhost:5000`. What is a likely reason?

- A. The server did not allow cross-origin requests
 - B. The CSS file is too large
 - C. The client used too many variables
 - D. `localhost` cannot be used in development
-

28. try/catch with await

Why is try/catch useful around `await fetch(...)`?

- A. It can handle errors during the asynchronous operation
 - B. It makes the request faster
 - C. It changes POST into GET
 - D. It automatically creates HTML elements
-

29. Status Code for Bad Input

A client sends missing or invalid data. Which status code is usually appropriate?

- A. 200 OK
 - B. 201 Created
 - C. 400 Bad Request
 - D. 404 Not Found
-

30. AI-Generated Code Review

A student submits this code:

```
app.post("/api/books", (req, res) => {  
  books.push(req.body);  
});
```

What is the main problem?

- A. The route modifies the array but never sends a response
- B. push cannot be used in Express
- C. POST routes cannot use `req.body`
- D. Arrays are not allowed on the server

Answers — Exam B

1. B — Visible page content belongs inside `<body>`.
2. B — `required` is the correct HTML attribute.
3. C — External JavaScript is connected with `<script src="..."></script>`.
4. A — The script may run before the DOM element is loaded.
5. A — Put the script at the end of `<body>` or use `defer`.
6. B — `.error` targets a class.
7. C — `#profile` targets an ID.
8. B — Later CSS with equal specificity overrides earlier CSS.
9. A — With `border-box`, the final width stays `300px`.
10. B — `flex-direction: column` arranges children vertically.
11. C — `==` allows type conversion, `===` does not.
12. B — `filter()` keeps values where the condition is true.
13. C — Template literals use backticks.
14. B — `addEventListener("click", callback)` is correct.
15. A — `JSON.stringify()` converts an object into a JSON string.
16. A — The client sends `POST`, but the server only accepts `GET`.
17. A — Every request should eventually receive a response.
18. B — `express.static("public")` serves static files.
19. B — Sensitive validation against a database belongs on the server.
20. A — `req.query.category` is `"games"`.
21. B — `:id` captures `15` as a route parameter.
22. A — `.env` often contains secrets or local configuration.
23. B — `.gitignore` tells Git which files/folders not to track.
24. B — `package.json` stores metadata, scripts, and dependencies.
25. B — `npm install` installs dependencies from `package.json`.
26. A — The `Content-Type` should be `application/json`.
27. A — Different ports are different origins; the server must allow `CORS`.
28. A — `try/catch` can handle errors from `async` code.
29. C — `400 Bad Request` fits invalid client input.
30. A — The server never sends a response, so the client may keep waiting.